

Python pour le trading — Document 11

Les lignes de commande avancées

1. Pourquoi la ligne de commande, et ses deux visages

Jusqu'ici, on a surtout lancé du code depuis un IDE. Mais dès qu'un programme quitte le poste de développement — planifié la nuit, déployé sur un serveur sans écran, enchaîné à d'autres — c'est la **ligne de commande** qui le pilote. La maîtriser, ce n'est pas une nostalgie : c'est la façon la plus directe, la plus scriptable et la plus reproductible de faire tourner un outil.

Ce module a deux visages complémentaires. Le premier est de **piloter le terminal** : savoir composer des commandes, rediriger des flux, enchaîner de petits outils, filtrer des fichiers de logs. Le second est d'**écrire ses propres outils en ligne de commande** avec Python — transformer un script en une commande propre, avec des options, une aide et un code de sortie. On apprend d'abord à consommer le terminal, puis à le nourrir.

Le fil conducteur reste le **gestionnaire d'ordres** des modules précédents : on l'imagine désormais lancé par une commande planifiée, écrivant ses logs dans un fichier, et piloté par des options (quel symbole, quel seuil, mode simulation ou réel).

***Pourquoi ça compte.** En production, personne ne clique sur « Run ». Une stratégie se lance par une commande, souvent sans interface : un serveur la planifie, redirige ses sorties vers des fichiers, et surveille son code de sortie. Le terminal est l'interface réelle entre ton code et le monde qui l'exécute.*

2. Anatomie d'une commande et code de sortie

Une commande suit toujours la même forme : un **programme**, suivi d'**arguments** (ce sur quoi il agit) et d'**options** (comment il agit, souvent préfixées de - ou --).

```
python strategie.py AAPL --seuil 5 --simulation # programme argument options
# \____ programme ____/ \arg/ \__ options ____/
```

Deux shells dominant, et ce module donne les deux à chaque fois : **PowerShell** (Windows, celui de ta machine) et **bash** (macOS / Linux, celui des serveurs). Les concepts sont identiques ; seule la syntaxe de quelques commandes change.

Chaque commande, en se terminant, renvoie un **code de sortie** : un entier où 0 signifie « succès » et tout le reste « échec ». C'est invisible à l'œil, mais c'est ce que lit le shell — et tout planificateur — pour savoir si l'étape a réussi. On le consulte juste après une commande :

```
python strategie.py AAPL      # on lance
echo $LASTEXITCODE           # PowerShell : 0 = succes, autre = echec
echo $?                       # bash       : idem
```

***Pourquoi ça compte.** Le code de sortie est le langage par lequel les programmes se disent « ça a marché » ou « ça a échoué ». C'est lui qui permet d'enchaîner : ne passer les ordres au marché que si le calcul du signal a réussi (code 0). Un script de trading qui échoue silencieusement en renvoyant 0 est un piège ; on y revient au volet Python.*

3. Les trois flux : entrée, sortie, erreur

Tout programme dispose de trois canaux standards : l'**entrée** (stdin), la **sortie** (stdout, les résultats normaux) et l'**erreur** (stderr, les messages d'anomalie). Les séparer permet de traiter les résultats et les erreurs différemment.

On **redirige** ces flux vers des fichiers avec des chevrons. > écrase, >> ajoute à la fin, et 2> cible spécifiquement les erreurs.

```
python strategie.py AAPL > signaux.log          # stdout -> fichier (ecrase)
python strategie.py AAPL >> signaux.log         # stdout -> ajoute a la fin
python strategie.py AAPL 2> erreurs.log        # stderr -> fichier a part
python strategie.py AAPL > out.log 2> err.log   # les deux, separement
```

Séparer stdout et stderr est précieux : le journal des signaux reste propre dans un fichier, tandis que les erreurs partent dans un autre qu'on surveille. À l'inverse, pour tout regrouper dans un même fichier, on redirige l'erreur vers la sortie :

```
python strategie.py AAPL > tout.log 2>&1       # bash : stderr suit stdout
python strategie.py AAPL *> tout.log           # PowerShell : tous les flux
```

Pourquoi ça compte. Un robot de trading tourne sans personne devant l'écran : ses sorties DOIVENT aller dans des fichiers. Garder les erreurs à part de la sortie normale, c'est pouvoir répondre à « la stratégie a-t-elle planté cette nuit ? » en lisant un seul fichier d'erreurs, sans fouiller des milliers de lignes de signaux.

4. Les tubes et le chaînage : composer de petits outils

La philosophie de la ligne de commande tient en une idée : de petits outils qui font une seule chose, reliés entre eux. Le **tube** (|) branche la sortie d'une commande sur l'entrée de la suivante, sans passer par un fichier intermédiaire.

```
# compter combien d'ordres d'ACHAT dans le journal :
cat signaux.log | grep ACHAT | wc -l          # bash
Get-Content signaux.log | Select-String ACHAT | Measure-Object -Line   # PowerShell
```

On **chaîne** aussi les commandes selon leur succès, en s'appuyant sur le code de sortie vu plus haut. && n'exécute la suite que si la précédente a réussi ; || seulement si elle a échoué ; ; enchaîne inconditionnellement.

```
python calcule_signal.py && python passe_ordres.py  # ordres SI signal OK
python strategie.py || python alerte.py             # alerte SI echec
git pull ; python strategie.py                      # l'un puis l'autre
```

Note : && et || existent en bash et dans PowerShell moderne (7+). Sous l'ancien Windows PowerShell 5.1, on chaîne plutôt avec if (\$?) { ... } pour tester le succès de la commande précédente.

Pourquoi ça compte. « Ne passer les ordres que si le signal a été calculé sans erreur » n'est pas une phrase abstraite : c'est exactement ce qu'exprime `calcule && passe`. Le chaînage conditionnel encode la logique de sécurité d'une chaîne de trading directement dans la ligne de commande.

5. Fichiers : sélection, recherche et filtrage

Manipuler des fichiers par lot repose sur le **globbing** : des jokers qui désignent plusieurs fichiers d'un coup. * remplace n'importe quelle suite de caractères, ? un seul.

```
ls *.log          # tous les fichiers .log (bash)
Get-ChildItem *.log # idem (PowerShell)
ls donnees_2026-*.csv # tous les CSV de 2026
```

Pour retrouver un fichier en profondeur dans une arborescence, on utilise la recherche récursive :

```
find . -name '*.parquet' # bash : tous les .parquet sous le dossier
Get-ChildItem -Recurse -Filter *.parquet # PowerShell : idem
```

5.1 Chercher DANS les fichiers : grep / Select-String

Le plus utile au quotidien est de fouiller le **contenu** des fichiers. `grep` (bash) et `Select-String` (PowerShell) affichent les lignes qui correspondent à un motif — au besoin une expression régulière (module 3).

```
grep 'ERREUR' *.log # lignes contenant ERREUR
grep -c 'ACHAT' signaux.log # combien de lignes ACHAT
grep -i -n 'timeout' erreurs.log # -i insensible casse, -n numero de ligne
```

```
Select-String 'ERREUR' *.log # equivalent PowerShell
Select-String -Pattern 'timeout' erreurs.log
```

Pourquoi ça compte. Un système de trading produit des journaux volumineux. Savoir extraire en une ligne « toutes les erreurs de timeout de la nuit » ou « tous les ordres rejetés sur AAPL » transforme une enquête d'une heure en une commande de dix secondes. C'est le réflexe de diagnostic de base sur un serveur.

6. Variables d'environnement et secrets

Une **variable d'environnement** est une valeur nommée que le shell transmet aux programmes qu'il lance. Elle sert à configurer un outil sans toucher à son code — et surtout à lui passer des **secrets** (clés d'API, mots de passe) sans les écrire en clair dans les fichiers.

```
$env:CLE_API = 'abc123' # définir (PowerShell, session courante)
export CLE_API=abc123 # définir (bash)
echo $env:CLE_API # lire (PowerShell)
echo $CLE_API # lire (bash)
```

Côté Python, on lit ces variables avec `os.environ` — jamais la clé n'apparaît dans le code :

```
import os
cle = os.environ['CLE_API'] # erreur nette si absente
cle = os.environ.get('CLE_API', '') # valeur par défaut si absente
```

Une variable spéciale mérite d'être comprise : **PATH**. C'est la liste des dossiers où le shell cherche les programmes. Quand python « n'est pas reconnu », c'est presque toujours que son dossier n'est pas dans le PATH — le même mécanisme d'interpréteur vu au module 10.

Pourquoi ça compte. Une clé d'API de courtier écrite en dur dans un script, puis versionnée par erreur, est une fuite qui peut coûter cher. Les variables d'environnement sont la manière standard de séparer les secrets du code : le même programme tourne en simulation ou en réel selon les variables qu'on lui donne, sans le modifier.

7. Du script à l'outil : sys.argv

On passe au second volet : **écrire** ses propres commandes. Un script Python reçoit ses arguments dans `sys.argv`, une liste de chaînes où le premier élément est le nom du script.

```
# strategie.py
import sys
print(sys.argv)
```

```
# python strategie.py AAPL --seuil 5
# -> ['strategie.py', 'AAPL', '--seuil', '5']
```

Cela fonctionne, mais tout est manuel : lire les positions, convertir les types (tout arrive en texte, '5' et non 5), gérer les options manquantes, écrire soi-même l'aide. Dès qu'un outil a plus d'un argument, `sys.argv` devient fragile. C'est précisément le problème que résout la bibliothèque standard suivante.

Pourquoi ça compte. Comprendre ``sys.argv`` démystifie ce qu'est « une commande » : rien de plus qu'un programme recevant une liste de mots. Mais le construire à la main ne passe pas l'échelle — d'où l'outil standard de la section suivante.

8. argparse : l'outil standard de la CLI

`argparse` est fourni avec Python. On y **déclare** les arguments et options attendus ; il se charge de les lire, de convertir les types, de valider, et de générer une aide `--help` complète — automatiquement.

```
# strategie.py
import argparse

p = argparse.ArgumentParser(description='Calcule un signal pour un symbole.')
p.add_argument('symbole', help='ex. AAPL')           # positionnel (requis)
p.add_argument('--seuil', type=float, default=5.0)    # option typée + défaut
p.add_argument('--simulation', action='store_true')   # drapeau vrai/faux
args = p.parse_args()

print(args.symbole, args.seuil, args.simulation)
```

L'appel devient lisible et robuste, et l'aide est offerte :

```
python strategie.py AAPL --seuil 2.5 --simulation
python strategie.py --help           # aide générée automatiquement
python strategie.py                  # erreur claire : 'symbole' manquant + code 2
```

8.1 Sous-commandes et code de sortie

Pour un outil qui fait plusieurs choses (à la manière de `git commit / git push`), on crée des **sous-commandes**. Et l'on renvoie explicitement un **code de sortie** avec `sys.exit` — 0 si tout va bien, non nul en cas d'échec, pour que le chaînage `&&` de la section 4 fonctionne.

```
sub = p.add_subparsers(dest='commande')
sub.add_parser('signal')           # python outil.py signal ...
sub.add_parser('ordres')           # python outil.py ordres ...

import sys
if erreur_de_connexion:
    print('connexion impossible', file=sys.stderr) # message sur stderr
    sys.exit(1)                                   # echec -> code 1
```

Pourquoi ça compte. Avec ``argparse``, ton script devient un vrai outil : typé, documenté, qui échoue proprement avec un code non nul et un message sur ``stderr``. C'est ce qui le rend sûr à planifier et à enchaîner — un maillon fiable d'une chaîne de trading, et non un script qu'on lance à la main en croisant les doigts.

9. click et typer : les CLI modernes

Au-delà de la bibliothèque standard, deux outils externes rendent l'écriture de CLI plus déclarative. **click** repose sur des décorateurs (module 3) ; **typer** (bâti sur click) déduit les arguments directement des annotations de type de la fonction — le typage du module 3 sert alors à définir la CLI.

```
# avec typer : la signature de la fonction EST l'interface
import typer
app = typer.Typer()

@app.command()
def signal(symbole: str, seuil: float = 5.0, simulation: bool = False):
    print(symbole, seuil, simulation)

if __name__ == '__main__':
    app()
```

Le même appel qu'avec `argparse` fonctionne, aide comprise, mais avec beaucoup moins de code. Le choix est un arbitrage : `argparse` ne demande rien à installer et suffit partout ; `click` / `typer` apportent confort et élégance au prix d'une dépendance externe (module 10).

***Pourquoi ça compte.** Pour un outil interne rapide, `argparse` suffit et ne pèse rien. Pour un outil que toute une équipe va utiliser et étendre, la lisibilité de `typer` réduit les erreurs. Savoir que les deux existent, et pourquoi choisir l'un plutôt que l'autre, vaut mieux que de les maîtriser tous les deux.*

10. Distribuer l'outil : de `python script.py` à une commande

Un outil abouti ne s'invoque plus par `python chemin/vers/script.py` : il devient une commande qu'on tape directement. Trois étapes, du plus simple au plus propre.

D'abord, **python -m** exécute un module par son nom, sans se soucier du chemin — utile dès qu'un projet est structuré en paquet :

```
python -m trading.strategie AAPL      # execute le module, ou qu'on soit
```

Ensuite, on déclare un **point d'entrée** dans le `pyproject.toml` (module 10) : une fois le projet installé, une vraie commande apparaît dans le PATH.

```
# pyproject.toml
[project.scripts]
strategie = 'trading.strategie:main'    # cree la commande 'strategie'
```

```
# puis, apres installation :
strategie AAPL --seuil 2.5                # plus besoin de 'python ...'
```

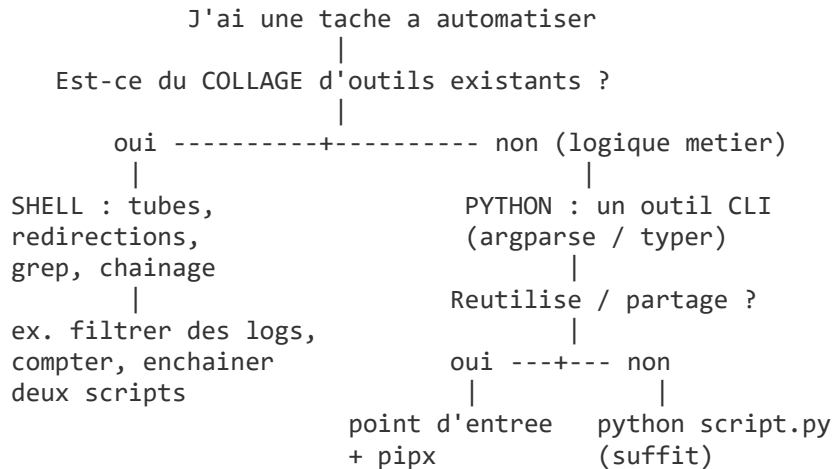
Enfin, pour distribuer un outil isolé de tout projet, **pipx** l'installe dans son propre environnement et l'expose comme commande globale — idéal pour un utilitaire maison partagé dans l'équipe.

```
pipx install ./mon-outil-trading             # commande dispo partout, env isole
```

***Pourquoi ça compte.** La différence entre un script qu'on est seul à savoir lancer et un outil que toute l'équipe utilise tient à cette dernière étape. Une commande installée, avec son nom propre et son aide, est ce qui fait passer un bout de code du statut de brouillon personnel à celui d'outil de production partagé.*

11. Fiche-synthèse : shell ou Python ?

Les deux volets se répondent. La question pratique — « pour cette tâche, j'écris une ligne de shell ou un outil Python ? » — se tranche simplement :



La règle : le **shell** excelle à **relier** des outils existants (filtrer un log, compter des lignes, enchaîner deux programmes sous condition). Dès qu'il y a de la **logique métier** — calculer un signal, décider d'un ordre — on écrit un **outil Python** propre, avec ses options et son code de sortie, qui devient à son tour un maillon qu'on branche dans le shell.

Ainsi bouclent les deux visages du module : on **pilote** le terminal pour orchestrer, et on l'**alimente** en outils Python fiables. Le gestionnaire d'ordres, lancé par une commande planifiée, redirigeant ses flux, chaîné sur son code de sortie et piloté par ses options, est désormais un citoyen à part entière de la ligne de commande.

Pourquoi ça compte. En trading comme ailleurs, la valeur n'est pas de tout faire en shell ni tout en Python, mais de savoir où passe la frontière : coller avec le shell, calculer avec Python. Cette division du travail est ce qui rend une chaîne d'exécution à la fois lisible, testable et robuste.